Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Table of Contents

1. CS-601 Data Security & Privacy Assignment: AI-Assisted Application Development, Data Security, Privacy and Cybersecurity Vulnerability Assessment 6

1. Assignment Title (Start 31 Aug 2026) 6

"Build It with AI, Break It Ethically, Secure It: Security and Privacy Evaluation of AI-Generated Applications" 6

2. Aim 6

3. Learning Objectives 7

LO1 AI-assisted software development 7
LO2 Understand AI-generated-code risk 7
LO3 Data security 7
LO4 Privacy 8
LO5 Vulnerability assessment 8
LO6 Ethical security testing 8
LO7 Vulnerability remediation 8
LO8 Security verification 8
LO9 Secure software lifecycle 9
LO10 Critical evaluation of AI 9

4. Recommended Assignment Architecture 9

Important restriction 9

5. Suggested Application Requirements 10

Functional components 10
Security-sensitive components 10

6. AI Usage Requirement 11

Sharing with you an Example AI prompt 11

7. Eight-Week Schedule (Starts from Sept 15, 2026) 11

Week 1 AI-Assisted Software Development and Threat Awareness (14th Sept 2026) 12

Theme "What can AI build?" and "Why shouldn't we trust it automatically?" 12
Activities 12
Security activity 12
Deliverables 12
Demonstration 13

8. Week 2 Understanding the AI-Generated Code (21 Sept 2026) 13

Theme 13

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

"Working code ≠ secure code." 13
Review areas 13
Deliverable 14

9. Week 3 Threat Modelling and Privacy Analysis (28 Sept 2026) ***14***

Theme **14**
"What are we protecting, from whom, and why?" 14
STRIDE 14
Example 15

Privacy assessment **15**
Data Flow Diagram 15
Deliverables 16

10. Week 4 Automated Security Assessment (05 Oct 2026) ***16***

Theme **16**
"Let the tools find what humans miss." 16
Recommended tools 16
Required output 17

11. Week 5 Controlled Vulnerability Demonstration (12 Oct 2026) ***17***

Theme **17**
"Show how the vulnerability can lead to data exposure." 17
Suggested vulnerabilities 17
Data-theft demonstration 18
Example 18
Deliverable 18

12. Week 6 Manual Remediation (19Oct2026) ***19***

Theme **19**
"Now fix the application." 19
Authentication 19
Input handling 19
Authorization 20
Secrets 20
Privacy 20
Deliverables 21

13. Week 7 Automated Remediation and Re-testing (19 Oct 2026) ***21***

Theme **21**
"Can tools help secure AI-generated code?" 21
Example process 21
Before remediation 22
After remediation 22

Required metrics **22**

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Important 22

14. Week 8 Final Security Audit and Presentation (26 Oct 2026) ***23***

Theme **23**

"Can you defend your application?" 23

Final demonstration (26th Oct 2026 Onwards) 23

15. Final Deliverables (Week of 26th Oct 2026) ***23***

Deliverable 1 Application **23**

Deliverable 2 AI Development Log **24**

Deliverable 3 Threat Model **24**

Deliverable 4 Privacy Assessment **24**

Deliverable 5 AI Code Security Assessment **25**

Deliverable 6 Security Tool Report **25**

Deliverable 7 Vulnerability Demonstration **25**

Deliverable 8 Final Secure Application **25**

Deliverable 9 Final Report **26**

16. Method for Evaluating AI-Generated Code ***26***

Layer 1 Functional correctness **27**

Layer 2 Secure coding review **27**

Layer 3 Static analysis **27**

Layer 4 Dependency analysis **27**

Layer 5 Dynamic security testing **28**

Layer 6 Privacy assessment **28**

Layer 7 Human security judgement **29**

17. Suggested Vulnerability Matrix ***29***

18. Real-World Incidents to Study ***30***

Case Study 1 Equifax, 2017 ..... **30**

Student question 30

19. Real-World Incident 2: Capital One, 2019 ..... ***30***

Student question 31

20. Real-World Incident 3 SolarWinds ***31***

Student question 31

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

21. Real-World Incident 4 MOVEit 31
Student question 32
22. Real-World Research: AI-Generated Code 32
23. Tools Recommended for the Assignment(July-Dec 2026 CS 601 Batch) .. 32
Code analysis 32
Semgrep 32
GitHub CodeQL 32
Bandit 33
Dependency security 33
OWASP Dependency-Check 33
pip-audit 33
npm audit 33
Dynamic application testing 33
OWASP ZAP 33
Secret detection 33
Gitleaks 33
Container security 33
Trivy 33
SBOM 34
Syft 34
Network/security observation 34
24. Assessment Rubric — 100 Marks (PI note: These will be deduced, as per the weightages suitable, post discussions in person) 34
25. Quality Criteria for Vulnerability Demonstrations 35
1. Existence 35
2. Reproducibility 35
3. Impact 35
4. Data relevance 35
5. Root cause 35
6. Mitigation 35
7. Verification 35
26. A Particularly Important Requirement: Compare AI and Human Security Judgement 35
Review A 36
Review B 36
Review C 36
27. AI Security Evaluation Score 36
AI Code Security Score (AICSS) 36

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

28. Suggested Secure Development Lifecycle (02 Nov 2026 onwards)..... 37

29. Ethical and Safety Rules 38

30. Expected Learning Outcomes 38

31. Final Student Reflection..... 39

32. Recommended Textbooks and Core References 39

Primary security references.....39

33. Standards and Professional References 40

A. NIST SP 800-218 — Secure Software Development Framework.....40
B. NIST SP 800-218A.....40
C. NIST DevSecOps Practices.....40
D. MITRE CWE 40
E. MITRE ATT&CK 40
F. CVE / NVD 40

34. Suggested Core Research Paper 41

35. Suggested Final Research Question..... 41

RQ1.....41
RQ2.....41
RQ3.....41
RQ4.....41
RQ5.....42
RQ6.....42
RQ7.....42
RQ8.....42

36. My Important Educational Message for you, as on in this semester ! 42

Message for you!

Dear Students, This Assignment, is deliberately such structured so that you do not merely “ask AI to write an application,” but instead learn the complete cycle as follows:

AI-assisted development → security assessment → vulnerability demonstration in a controlled lab → privacy/data-theft analysis → manual remediation → automated remediation → re-testing → secure-development reporting.

NOTE that, (1) The design aligns well with the NIST Secure Software Development Framework (SSDF), OWASP application-security guidance, and NIST's dedicated guidance for generative-AI software development. (2) NIST explicitly emphasizes that

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

AI-generated content should be monitored and validated by humans rather than trusted blindly. (NIST Computer Security Resource Center)

1. **CS-601 Data Security & Privacy Assignment: AI-Assisted Application Development, Data Security, Privacy and Cybersecurity Vulnerability Assessment**
2. **Duration:** 8 weeks
   Level: Postgraduate Engineering
   Team size: 2-3 students
   Suggested workload: 6-8 hours/week/student
   Assessment: ___ marks
   Application domain: Student-selected, subject to faculty approval

1. Assignment Title (Start 31 Aug 2026)

"Build It with AI, Break It Ethically, Secure It: Security and Privacy Evaluation of AI-Generated Applications"

2. Aim

To enable students to understand, implement and critically evaluate the **security and privacy implications of AI-assisted software development** by developing an application substantially using generative AI, systematically identifying vulnerabilities in the AI-generated code, demonstrating those vulnerabilities in an isolated and authorized laboratory environment, and applying manual and automated security controls to produce a hardened version.

The central learning philosophy is:

AI can accelerate software development, but AI-generated code must be treated as untrusted code until it has been reviewed, tested and secured.

This is particularly important because empirical research has demonstrated that AI coding assistants can produce insecure code. An NYU study involving 89 scenarios and roughly 1,700 generated programs found approximately 40% of the generated programs vulnerable in the scenarios tested. This should be treated as evidence of risk—not as a universal "40% vulnerability rate" for all AI-generated software. (NYU Center for Cyber Security)

With this assignment: Reflection is important because the objective is **not to educate students to distrust AI**. It is to educate my students to use AI **with engineering judgement and security assurance**.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

3. Learning Objectives

At the end of eight weeks, students should be able to:

LO1 AI-assisted software development

Explain and demonstrate how generative AI can assist with:

- requirements analysis
- architecture
- database design
- programming
- API development
- testing
- documentation
- debugging
- security analysis

LO2 Understand AI-generated-code risk

Identify why AI-generated code may contain:

- insecure defaults
- authentication weaknesses
- authorization flaws
- injection vulnerabilities
- improper input validation
- weak cryptography
- hard-coded secrets
- insecure dependency choices
- privacy violations
- excessive data collection
- insecure logging
- poor error handling

LO3 Data security

Assess how application design can expose:

- personally identifiable information (PII)
- credentials
- authentication tokens
- database records
- uploaded documents
- API keys
- session information

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- confidential business information

LO4 Privacy

Evaluate:

- what personal data the application collects
- why it collects it
- where it is stored
- who can access it
- whether it is retained unnecessarily
- whether it is exposed through logs/APIs
- whether data is encrypted
- whether third-party AI services receive sensitive information

LO5 Vulnerability assessment

Use a combination of:

human review + static analysis + dependency analysis + dynamic testing + security testing + threat modelling

to assess AI-generated applications.

LO6 Ethical security testing

Demonstrate vulnerabilities only in:

- the students' own application,
- intentionally vulnerable applications,
- isolated laboratory environments, or
- systems for which explicit authorization exists.

LO7 Vulnerability remediation

Apply security controls manually and through appropriate tools and demonstrate that the vulnerability has been reduced or eliminated.

LO8 Security verification

Understand & Demonstrate that:

Fixing code is not the same as proving that the vulnerability is fixed.

Students must therefore perform **re-testing after remediation**.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

LO9 Secure software lifecycle

Map their development process to a secure software development lifecycle and NIST SSDF principles. NIST's SSDF organizes secure development around preparing the organization, protecting software, producing well-secured software and responding to vulnerabilities. (NIST Computer Security Resource Center)

LO10 Critical evaluation of AI

Students should be able to answer:

"How do I know whether the code generated by AI is safe?"

rather than merely:

"Can AI generate working code?"

4. Recommended Assignment Architecture

Group of Students- select **one realistic application**.

Suitable examples include:

1. Student information portal
2. Hospital appointment application
3. E-commerce application
4. Online examination system
5. Digital library
6. Expense-management application
7. Employee leave-management system
8. Research-document management system
9. Online complaint-management system
10. IoT monitoring dashboard
11. Secure file-sharing application
12. Personal-finance application
13. Academic project-management system
14. Small banking-like transaction simulator
15. AI-powered document/question-answering application

Important restriction

Students **must not use real personal, financial, medical, institutional or confidential data**.

Use synthetic data such as:

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Alice Student
alice@example.test
Student ID: STU001
Phone: 9999999999

The application should contain enough sensitive-looking data to study privacy and security, but no actual sensitive information.

5. Suggested Application Requirements

Every project should contain, where appropriate:

Functional components

- user registration
- login/logout
- role-based access
- database
- CRUD operations
- search
- file upload or document handling
- REST API
- logging
- administrator functionality
- password management
- optional AI feature

Security-sensitive components

At least **six** of the following should be incorporated:

Component	Security learning
Login	Authentication
Roles	Authorization
Database	SQL injection/data protection
Search	Injection/input validation
File upload	Untrusted input
Password storage	Cryptography
Sessions	Session security
API	API authorization
Logs	Privacy/information leakage
Cloud configuration	Misconfiguration

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Component	Security learning
Dependencies	Supply-chain security
AI API	AI/data privacy
Document retrieval	Prompt/data leakage
Admin functions	Privilege escalation

6. AI Usage Requirement

AI must be an **integral part of the assignment**.

Students should use a generative-AI coding assistant to produce substantial portions of the application.

For every important AI interaction, students should preserve:

- prompt
- AI response
- generated code
- subsequent modification
- reason for modification
- security implications

IMPORTANT- Thus the project contains an **AI development audit trail**.

Sharing with you an Example AI prompt

> "Generate a Python Flask REST API for a student management application with registration, login, student profiles and administrator functionality using a relational database."
>
> Students then **must not assume that the resulting code is secure**.
>
> They should ask a second question:
>
> "Review this code for security vulnerabilities."

But the important educational point is that **the AI's own security review is only one input into the assessment**.

7. Eight-Week Schedule (Starts from Sept 15, 2026)

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Week 1 AI-Assisted Software Development and Threat Awareness (14th Sept 2026)

Theme "What can AI build?" and "Why shouldn't we trust it automatically?"

Activities

Students:

1. Select an application.
2. Define requirements.
3. Create architecture.
4. Identify data assets.
5. Identify users and roles.
6. Use AI to generate an initial application.
7. Record all significant prompts.
8. Run the application.

Security activity

Create a preliminary:

Asset–Threat–Vulnerability–Impact table

Example:

Asset	Threat	Vulnerability	Impact
Student records	Unauthorized access	Broken authorization	Privacy breach
Passwords	Credential theft	Weak password storage	Account compromise
Database	Injection	Unsanitized input	Data theft
API	Abuse	Missing authorization	Unauthorized modification
Files	Information disclosure	Insecure upload/download	Data exposure

Deliverables

- Project proposal
- Requirements specification
- Architecture diagram
- Data-flow diagram
- Data inventory
- Initial threat model
- AI prompt log

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- AI-generated application- Version 0

Demonstration

5-10 minute demo:

“Here is the application AI helped us build.”

8. Week 2 Understanding the AI-Generated Code (21 Sept 2026)

Theme

“Working code ≠ secure code.”

Students conduct a manual code review.

Review areas

Authentication

- password handling
- password reset
- session management
- login failure handling

Authorization

- role checks
- object-level authorization
- administrator functions

Input validation

- user input
- URL parameters
- HTTP headers
- JSON
- uploaded files

Database

- query construction
- ORM usage
- SQL statements

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- database permissions

Secrets

Search for:

- passwords
- API keys
- tokens
- encryption keys
- credentials

Error handling

Check whether errors expose:

- database information
- stack traces
- file paths
- credentials
- internal architecture

Deliverable

AI-Generated Code Security Review Report- Version 1

At least **15 security observations** should be recorded, even if some turn out to be false positives.

9. Week 3 Threat Modelling and Privacy Analysis (28 Sept 2026)

Theme

"What are we protecting, from whom, and why?"

Students develop a formal threat model.

Suggested methodology:

STRIDE

- Spoofing
- Tampering
- Repudiation

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- Information disclosure
- Denial of service
- Elevation of privilege

Map threats to the application.

Example

Threat	**STRIDE category**	**Asset**	**Vulnerability**	**Risk**
Student views another student's record	Information disclosure	PII	IDOR/BOLA	High
User modifies another account	Tampering	Account	Broken authorization	High
Attacker submits malicious query	Injection	Database	SQL injection	Critical
Developer credential exposed	Information disclosure	Secret	Hard-coded key	Critical

Privacy assessment

Students create a:

Data Flow Diagram

Show:

User
↓
Web UI
↓
API
↓
Application
↓
Database
↓
Logs / Backups / External Services

If AI services are used:

User
↓
Application

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

↓
AI Service
↓
Model

Students must ask/check/understand/evaluate:

- Does personal data reach the AI service?
- Is sensitive information included in prompts?
- Is unnecessary data transmitted?
- Is the information retained?
- Are secrets accidentally included?
- Are logs capturing personal information?
- Is data encrypted in transit?
- Is data encrypted at rest?

Deliverables

- STRIDE threat model
- Data-flow diagram
- Privacy impact assessment
- Data classification table
- Threat register

10. Week 4 Automated Security Assessment (05 Oct 2026)

Theme

“Let the tools find what humans miss.”

Students use several classes of security tools.

Recommended tools

Tool category	Example tools/ use as suitable	Purpose
SAST	Semgrep, Bandit, CodeQL	Source-code vulnerabilities
Dependency scanning	OWASP Dependency-Check, npm audit, pip-audit	Vulnerable libraries
Secret scanning	Gitleaks, TruffleHog	Credentials/secrets
DAST	OWASP ZAP	Web application testing

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Tool category	Example tools/ use as suitable	Purpose
Container scanning	Trivy	Container/image vulnerabilities
Infrastructure scanning	Checkov	IaC misconfiguration
Fuzzing	AFL++, libFuzzer or language-specific tools	Unexpected inputs
SBOM	Syft	Software-component inventory
Vulnerability database	NVD/CVE	Vulnerability identification

NOTE- Students do **not** need to use every tool. A minimum of:

one SAST + one dependency scanner + one DAST + one secret scanner

is recommended.

Required output

For every tool:

Tool Finding Severity Location True/False Positive Evidence

11. Week 5 Controlled Vulnerability Demonstration (12 Oct 2026)

Theme

“Show how the vulnerability can lead to data exposure.”

This is the most important practical week.

Students demonstrate selected vulnerabilities against their own application or a deliberately vulnerable laboratory environment.

Suggested vulnerabilities

Mapped to OWASP Top 10:

- Broken access control
- Cryptographic failures
- Injection
- Insecure design

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- Security misconfiguration
- Vulnerable/outdated components
- Authentication failures
- Software/data integrity failures
- Logging/monitoring failures
- SSRF

These categories are directly aligned with the OWASP Top 10 application-security awareness framework. (OWASP Foundation)

Data-theft demonstration

The demonstration should show a controlled chain such as:

Vulnerable Input
↓
Security Weakness
↓
Unauthorized Action
↓
Access to Synthetic Data
↓
Evidence Captured
↓
Impact Explained

Do not demonstrate attacks against external systems.

Example

Instead of stealing real data:

Database
---
student001
student002
student003

The demonstration proves that an unauthorized test user can access records they should not be able to access.

Deliverable

Vulnerability Demonstration Video / Live Demonstration

For every vulnerability:

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

1. vulnerability description
2. vulnerable code
3. security principle violated
4. controlled demonstration
5. evidence
6. potential impact
7. affected data
8. severity
9. mitigation strategy

12. Week 6 Manual Remediation (19Oct2026)

Theme

“Now fix the application.”

Students manually modify the application.

Examples:

Authentication

Before:

Password → insecure storage

After:

Password
↓
Strong password hashing
↓
Stored verifier

Input handling

Before:

User input → database query

After:

User input
↓

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Validation
↓
Parameterized query / safe ORM
↓
Database

Authorization

Before:

User → /record/123
↓
Data

After:

User
↓
Authenticate
↓
Authorize
↓
Verify ownership/role
↓
Data

Secrets

Before:

API_KEY = "..."

After:

Environment/secret-management mechanism

Privacy

Before:

Application
↓
Everything
↓
Logs

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

After:

Application
↓
Minimum necessary information
↓
Privacy-aware logs

Deliverables

- Secure Version 1
- Before/after code
- Remediation table
- Explanation of each fix
- Security-control mapping

13. Week 7 Automated Remediation and Re-testing (19 Oct 2026)

Theme

"Can tools help secure AI-generated code?"

Students use AI and security tools again—but this time with a secure-development methodology.

Example process

AI-generated code
↓
Human review
↓
SAST
↓
Dependency scan
↓
DAST
↓
Secret scan
↓
Threat model
↓
Manual remediation
↓
AI-assisted remediation

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

↓
Security testing again
↓
Final application

Students compare:

Before remediation

versus

After remediation

using measurable results.

Required metrics

Students should report:

Metric	Before	After
SAST findings		
High severity findings		
Critical findings		
Dependency vulnerabilities		
Secrets detected		
DAST findings		
Authentication weaknesses		
Authorization weaknesses		
Privacy issues		

Important

Students must distinguish:

- vulnerability removed
- vulnerability mitigated
- false positive
- accepted risk
- unresolved issue

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

14. Week 8 Final Security Audit and Presentation (26 Oct 2026)

Theme

“Can you defend your application?”

Students perform a final independent security review.

Please should attempt to answer:

If I were the attacker, what would I target?

and:

If I were the security engineer, how would I prevent it?

Final demonstration (26th Oct 2026 Onwards)

Each team gets approximately: **15–20 minutes**

to demonstrate:

1. Application functionality
2. AI-assisted development
3. Original vulnerable implementation
4. Vulnerability
5. Controlled exploitation
6. Data/privacy impact
7. Security tools used
8. Manual remediation
9. AI-assisted remediation
10. Re-testing
11. Final secure implementation
12. Lessons learned

15. Final Deliverables (Week of 26th Oct 2026)

Each group submits a complete **Security Engineering Portfolio**.

Deliverable 1 Application

- source code
- README
- setup instructions

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- database
- test data
- configuration

Deliverable 2 AI Development Log

For each significant AI interaction:

Date Objective Prompt AI output Student modification Security consequence

This prevents students from treating AI as a black box.

Deliverable 3 Threat Model

Include:

- assets
- actors
- trust boundaries
- attack surfaces
- threats
- mitigations
- residual risks

Deliverable 4 Privacy Assessment

Include:

- data inventory
- data classification
- collection purpose
- storage location
- retention
- access controls
- transmission
- third-party sharing
- logging
- deletion
- privacy risks

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Deliverable 5 AI Code Security Assessment

For each finding:

Finding ID
Vulnerability
Location
Evidence
Root cause
Threat
Affected asset
Impact
Likelihood
Severity
Recommended mitigation
Actual mitigation
Verification method
Status

Deliverable 6 Security Tool Report

Include:

- tool
- version
- configuration
- scan date
- findings
- false positives
- remediation
- re-scan results

Deliverable 7 Vulnerability Demonstration

For each selected vulnerability:

Before → Vulnerability → Controlled demonstration → Impact → Fix → Re-test

Deliverable 8 Final Secure Application

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

The final version must demonstrate measurable improvement.

Deliverable 9 Final Report

Suggested length:

35–50 pages excluding appendices

Recommended structure:

1. Abstract
2. Introduction
3. Problem statement
4. Objectives
5. Application architecture
6. AI development methodology
7. AI prompt methodology
8. Threat model
9. Privacy analysis
10. Security assessment methodology
11. Vulnerabilities discovered
12. Vulnerability demonstrations
13. Data-security impact
14. Privacy impact
15. Manual remediation
16. AI-assisted remediation
17. Automated security testing
18. Before/after comparison
19. Real-world incident comparison
20. Limitations
21. Ethical considerations
22. Lessons learned
23. Conclusion
24. References
25. Appendices

16. Method for Evaluating AI-Generated Code

This should be a **central component of the assignment**, not an optional activity.

Students should use a seven-layer evaluation methodology.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Layer 1 Functional correctness

Ask:

Does the program actually do what was requested?

Layer 2 Secure coding review

Ask:

Is the implementation secure?

Review:

- authentication
- authorization
- validation
- cryptography
- error handling
- session management
- secrets
- file handling
- database access

Layer 3 Static analysis

Run SAST tools.

Purpose:

Identify suspicious or vulnerable source-code patterns without executing the application.

Layer 4 Dependency analysis

Build an inventory of third-party components.

Ask:

- Which libraries are being used?

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- Which versions?
- Are any vulnerable?
- Are dependencies unnecessarily included?
- Are transitive dependencies safe?

This is especially important because AI may automatically introduce libraries that the developer did not originally plan to use.

Layer 5 Dynamic security testing

Run the application and test its externally observable behaviour.

Examples:

- authentication testing
- authorization testing
- input validation
- session testing
- API testing
- error handling
- file handling

Layer 6 Privacy assessment

Evaluate:

What data does this application expose even when it is not technically “hacked”?

For example:

Sensitive data
↓
Application
↓
Logs
↓
Debug messages
↓
Third-party API
↓
Backups

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

A privacy problem can therefore exist even without a conventional vulnerability.

Layer 7 Human security judgement

Finally ask:

Would I deploy this application in an organization?

The answer must be justified.

NIST's DevSecOps guidance specifically emphasizes that AI-generated content requires human monitoring and validation and that organizations need verifiable processes to prevent insecure or non-functional code from entering development. (NIST Pages)

17. Suggested Vulnerability Matrix

Students can use the following matrix.

Vulnerability	Security principle	Data-security impact	Typical mitigation
SQL injection	Input integrity	Database disclosure/modification	Parameterized queries
Broken access control	Least privilege	Unauthorized PII access	Server-side authorization
Weak password storage	Confidentiality	Credential compromise	Strong password hashing
Hard-coded secret	Confidentiality	API/database compromise	Secret management
Excessive logging	Privacy	PII disclosure	Data minimization
Weak encryption	Confidentiality	Data exposure	Modern cryptography
Insecure file upload	Integrity/confidentiality	Malicious files/data exposure	Validation + isolation
Debug mode	Confidentiality	Internal information disclosure	Disable in production
Vulnerable dependency	Supply chain	Application compromise	Patch/update/SBOM

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Vulnerability	Security principle	Data-security impact	Typical mitigation
Missing API authorization	Authorization	Unauthorized records	Object-level authorization
SSRF	Network trust	Internal-service access	URL validation/network controls
Insecure CORS	Access control	Unauthorized API access	Restrictive CORS
Poor session handling	Authentication	Account takeover	Secure session controls

---

18. Real-World Incidents to Study

The assignment should connect laboratory findings to actual incidents.

Case Study 1 Equifax, 2017

Equifax's breach is an excellent case study for:

- vulnerable components
- patch management
- vulnerability management
- web application security
- PII exposure
- security monitoring

Equifax reported that the attack vector involved **CVE-2017-5638 in Apache Struts**, and that personal information relating to approximately 143 million U.S. consumers was potentially affected. (Equifax Inc.)

Student question

If an AI-generated application automatically introduced an outdated vulnerable dependency, what controls would have prevented an Equifax-like failure?

19. Real-World Incident 2: Capital One, 2019

Useful for studying:

- cloud security
- configuration errors
- access control

- data exposure
- continuous security monitoring

Capital One reported unauthorized access to customer/applicant information through a configuration vulnerability and subsequently described augmenting automated scanning to continuously look for the issue. (Capital One)

Student question

Can an application be secure at the source-code level but insecure because of deployment configuration?

Expected answer: If **Yes/No**

Justify.
This is a crucial lesson.

20. Real-World Incident 3 SolarWinds

Useful for studying:

- software supply-chain security
- trusted software
- build integrity
- third-party dependencies
- malicious code insertion
- detection and response

CISA documented the SolarWinds Orion supply-chain compromise and active exploitation affecting multiple organizations. (cisa.gov)

Student question

If AI automatically adds third-party libraries, how can we establish software provenance and trust?

Learn- How this connects directly to modern secure software development.

21. Real-World Incident 4 MOVEit

MOVEit provides a useful case for:

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- web application vulnerabilities
- mass exploitation
- sensitive-data theft
- vulnerability management
- patching
- incident response

CISA/FBI issued guidance concerning exploitation of a MOVEit Transfer vulnerability associated with data theft. (cisa.gov)

Student question

How quickly should a vulnerable application component be identified, assessed, patched and re-tested?

22. Real-World Research: AI-Generated Code

The assignment should explicitly discuss the NYU empirical research on GitHub Copilot.

The researchers generated 1,692 programs across 89 scenarios and reported approximately 40% vulnerable in their experimental setting. (NYU Center for Cyber Security)

This provides an excellent academic basis for the assignment's central hypothesis:

AI-generated code should undergo the same- or stronger- security validation as human-generated code.

Students should critically evaluate the research rather than simply repeating the 40% statistic.

23. Tools Recommended for the Assignment(July-Dec 2026 CS 601 Batch)

Code analysis

Semgrep

Pattern-based static security analysis.

GitHub CodeQL

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Semantic code-security analysis.

Bandit

Python security analysis.

Dependency security

OWASP Dependency-Check

Useful for identifying vulnerable third-party dependencies.

pip-audit

Useful for Python dependencies.

npm audit

Useful for Node.js projects.

Dynamic application testing

OWASP ZAP

Students can use it against their own laboratory application.

Secret detection

Gitleaks

Useful for finding accidentally committed secrets.

Container security

Trivy

Useful for:

- container images
- packages

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- dependencies
- configuration

SBOM

Syft

Useful for generating a Software Bill of Materials.

Network/security observation

Depending on course infrastructure:

- Wireshark
- browser developer tools
- application logs
- local proxy tools

The tools should be used to **understand and verify vulnerabilities**, not simply to generate a list of scanner findings.

24. Assessment Rubric — 100 Marks (PI note: These will be deduced, as per the weightages suitable, post discussions in person)

Component	Marks
AI-assisted application development	10
AI prompt/documentation discipline	5
Threat modelling	10
Privacy/data-flow analysis	10
Manual code security assessment	10
Automated security testing	10
Vulnerability demonstrations	15
Manual remediation	10
Automated/AI-assisted remediation	5
Before/after verification	5
Real-world case-study analysis	5
Final presentation/viva	5

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Component	Marks
Total	**100**

25. Quality Criteria for Vulnerability Demonstrations

A student should **not receive full marks merely because a scanner reports a vulnerability**.

For full credit, the student should establish:

1. Existence

The vulnerability is actually present.

2. Reproducibility

The vulnerability can be reproduced in the authorized laboratory.

3. Impact

The student can explain what an attacker could gain.

4. Data relevance

The student identifies what information could be exposed.

5. Root cause

The student identifies why the AI-generated implementation was vulnerable.

6. Mitigation

A technically appropriate fix is implemented.

7. Verification

The student demonstrates that the vulnerability is no longer exploitable or that residual risk is understood.

26. A Particularly Important Requirement: Compare AI and Human Security Judgement

Students should conduct three reviews:

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

Review A

AI-generated code → AI security review

Review B

AI-generated code → human security review

Review C

AI-generated code → security tools

Then compare:

Finding	AI review	Human review	Tool	Actually vulnerable?
SQL injection	Yes	Yes	Yes	Yes
Authorization flaw	No	Yes	Sometimes	Yes
Hard-coded secret	Yes	Yes	Yes	Yes
Privacy issue	No	Yes	No	Yes
False positive	Yes	No	Yes	No

NOTE- I convey, this is the MOST educational components of the course.

27. AI Security Evaluation Score

Students can create a simple:

AI Code Security Score (AICSS)

For example:

$$AICSS = 100 - (W_c + W_h + W_m + W_p + W_d)$$

where:

- W_c = critical vulnerability penalty
- W_h = high vulnerability penalty
- W_m = medium vulnerability penalty
- W_p = privacy-risk penalty
- W_d = dependency/supply-chain penalty

The exact weighting should be defined upon discussions with me (the instructor).

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

> NOTE: More importantly, you need to understand that: **A numerical score is not proof of security.** It is only a comparative educational metric.

28. Suggested Secure Development Lifecycle (02 Nov 2026 onwards)

The final assignment should require students to demonstrate:

Requirements
↓
Threat Model
↓
Architecture
↓
AI-assisted Coding
↓
Human Code Review
↓
SAST
↓
Dependency Analysis
↓
Secret Scanning
↓
Build/Test
↓
DAST
↓
Vulnerability Assessment
↓
Manual Remediation
↓
AI-assisted Remediation
↓
Re-test
↓
Security Sign-off
↓
Deployment
↓
Monitoring

This is consistent with the secure-development philosophy of NIST SSDF and current NIST DevSecOps guidance. (NIST Computer Security Resource Center)

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

29. Ethical and Safety Rules

This should be **mandatory** in the assignment document.

Students shall:

- test only applications they own or are explicitly authorized to test;
- use synthetic data;
- not attack public websites;
- not scan university systems without written authorization;
- not use real credentials;
- not intentionally expose real personal information;
- not publish secrets/API keys;
- not conduct denial-of-service attacks;
- not attempt unauthorized persistence;
- not exfiltrate real data;
- document the laboratory boundaries.

The goal is:

Learn how attacks work by safely reproducing security failures, not by attacking real systems.

30. Expected Learning Outcomes

At the end of the assignment, a successful student should be able to say: **"I can use AI to build software."**

But, more importantly: **"I know that generated software is not automatically secure."**

They should also be able to say:

- I can identify security-sensitive AI-generated code.
- I can distinguish authentication from authorization.
- I can identify common application vulnerabilities.
- I can recognize data-security risks.
- I can identify privacy risks that scanners may miss.
- I can perform basic threat modelling.
- I can use SAST.
- I can use DAST.
- I can scan dependencies.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

- I can detect secrets.
- I can interpret CVEs.
- I can assess false positives.
- I can demonstrate vulnerabilities ethically.
- I can manually remediate vulnerabilities.
- I can use AI to assist remediation without blindly trusting it.
- I can re-test after remediation.
- I can explain residual risk.
- I can evaluate whether an AI-generated application is ready for deployment.

31. Final Student Reflection

Every student should answer these questions individually:

1. **What did AI do better than you?**
2. **What did AI do worse than you?**
3. **Which vulnerability did AI fail to identify?**
4. **Which vulnerability did security tools fail to identify?**
5. **Which vulnerability did human review identify?**
6. **Did AI-generated remediation always work?**
7. **Did an apparently secure fix introduce another problem?**
8. **What privacy issue was not a conventional cybersecurity vulnerability?**
9. **What would you change before deploying this application commercially?**
10. **Would you trust AI-generated code in a safety/security-critical application? Why or why not?**

This reflection is important because the objective is **not to teach students to distrust AI**. It is to teach them to use AI **with engineering judgement and security assurance**.

32. Recommended Textbooks and Core References

Primary security references

1. IMPORTANT: All my classroom discussions, and the reading assignments referred, time to time. Relate and connect the dots!
2. **William Stallings & Lawrie Brown — *Computer Security: Principles and Practice***
   Excellent foundation for confidentiality, integrity, authentication, access control, cryptography and security architecture.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

3. **Michael E. Whitman & Herbert J. Mattord — *Principles of Information Security***
   Useful for information-security management and security principles.
4. **Ross Anderson — *Security Engineering***
   Particularly valuable for understanding security as a systems-engineering problem rather than merely a collection of vulnerabilities.
5. **OWASP Web Security Testing Guide**
   Recommended practical reference for application-security testing.
6. **OWASP Top 10**
   Use for application-security risk classification. (OWASP Foundation)

33. Standards and Professional References

A. NIST SP 800-218 — Secure Software Development Framework
   a. Use this as the principal secure-development framework. (NIST Computer Security Resource Center)

B. NIST SP 800-218A
   a. Especially important for this assignment because it specifically extends SSDF with practices for **generative AI and dual-use foundation models**. (NIST)

C. NIST DevSecOps Practices
   a. Useful for connecting:
   b. security automation
   c. CI/CD
   d. SAST
   e. vulnerability management
   f. continuous monitoring
   g. AI-assisted software development
   h. NIST explicitly describes the use of AI for coding, security analysis, vulnerability detection and remediation while emphasizing human validation. (NIST Pages)

D. MITRE CWE
   Use for classifying weaknesses such as:

   a. CWE-79 — Cross-Site Scripting
   b. CWE-89 — SQL Injection
   c. CWE-287 — Improper Authentication
   d. CWE-862 — Missing Authorization
   e. CWE-798 — Hard-coded Credentials

E. MITRE ATT&CK
   a. Use for understanding attacker behaviour and mapping appropriate threats.

F. CVE / NVD
   a. Use for dependency and component vulnerability research.

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

34. Suggested Core Research Paper

A particularly relevant research paper for this assignment is:

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. — "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions."

The research investigated AI-generated code under security-sensitive scenarios and found substantial vulnerability rates in its experimental conditions. (DOI) https://dl.acm.org/doi/10.1145/3610721

Students required to **critically review this paper**, especially:

- experimental design
- threat scenarios
- vulnerability classification
- limitations
- generalizability
- implications for software engineering

35. Suggested Final Research Question

To make the assignment meaningful/impactful for you, every team should investigate one research question.

Examples:

RQ1

Does AI-generated code contain more security weaknesses when the prompt lacks explicit security requirements?

RQ2

Does explicitly asking an AI coding assistant for secure code reduce vulnerabilities?

RQ3

Which vulnerabilities are most frequently missed by AI-generated applications?

RQ4

Which vulnerabilities are detected by SAST but missed by human reviewers?

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

RQ5

Which privacy risks cannot be detected effectively by conventional vulnerability scanners?

RQ6

Does AI-assisted vulnerability remediation introduce new vulnerabilities?

RQ7

How effective is a combination of AI review + SAST + DAST compared with human review alone?

RQ8

How does adding threat modelling before AI coding affect the security of the resulting application?

---

36. My Important Educational Message for you, as on in this semester !

I would make this the **official learning (statement printed on the assignment sheet, if the need be!)**:

Generative AI is a software-development accelerator, not a security authority.

A program that compiles is not necessarily correct.
A program that works is not necessarily secure.
A program that passes functional tests is not necessarily privacy-preserving.
A program that passes an AI security review is not necessarily safe.
A program that passes one security scanner is not necessarily secure.

For DATA SECURITY & PRIVACY, the Secure software development requires threat modelling, human judgement, automated analysis, controlled security testing, remediation and independent re-verification. NEW Software Engg perspective!

This principle will help you understand that this assignment (and everyone's efforts) substantially more valuable than a conventional "develop an application using ChatGPT/Copilot" project.

It also reflects the direction of current NIST guidance: generative AI should be incorporated into secure development processes, while human oversight, validation and verifiable security processes remain essential. (NIST)

Dear Students, Subject CS-601; Academic Batch 2026-28; Term July-Dec 2026

Subject OIC: Manisha J. Nene; mjnene@diat.ac.in

Discussed & Floated:31 Aug 2026

This be considered as Mini Project / Assignment:
"AI-Assisted Secure Software Engineering: Development, Vulnerability Assessment, Data Security and Privacy"

Eight-week progression:
Build → Understand → Threat-model → Scan → Break ethically → Fix → Re-test → Defend

OUTCOMES, as subject0in-charge, I look forward for is that this structure gives you the **software-engineering skills, AI literacy, cybersecurity skills, privacy awareness, vulnerability-assessment skills and secure-development practices in one integrated assignment**, rather than treating AI and cybersecurity as two unrelated topics.

END

Write to mjnene@diat.ac.in/ discuss with me in person to clarify your doubts!

All the best, happy learning!